In [6]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load the data into a pandas DataFrame
file_path = 'sales_data.csv'
df = pd.read_csv(file_path)

# Display the first few rows of the DataFrame
print(df.head())

# Check for missing values
print(df.isnull().sum())

# Fill or drop missing values if necessary
df['Sales'].fillna(df['Sales'].mean(), inplace=True)
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)

# Summary statistics
print(df.describe())

# Group by product and calculate the total sales and quantity
product_summary = df.groupby('Product').agg({
    'Sales': 'sum',
    'Quantity': 'sum'
}).reset_index()

print(product_summary)

# Bar plot of total sales by product
plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()

# Line plot of sales over time
df['Date'] = pd.to_datetime(df['Date'])

sales_over_time = df.groupby('Date').agg({
    'Sales': 'sum'
}).reset_index()

plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'], sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.show()

# Pivot table to analyze sales by region and product
pivot_table = df.pivot_table(
    values='Sales',
```

```
    index='Region',
    columns='Product',
    aggfunc='sum',
    fill_value=0
)

print(pivot_table)

# Correlation matrix
correlation_matrix = df[['Sales', 'Quantity']].corr()

print(correlation_matrix)

# Heatmap of the correlation matrix
import seaborn as sns

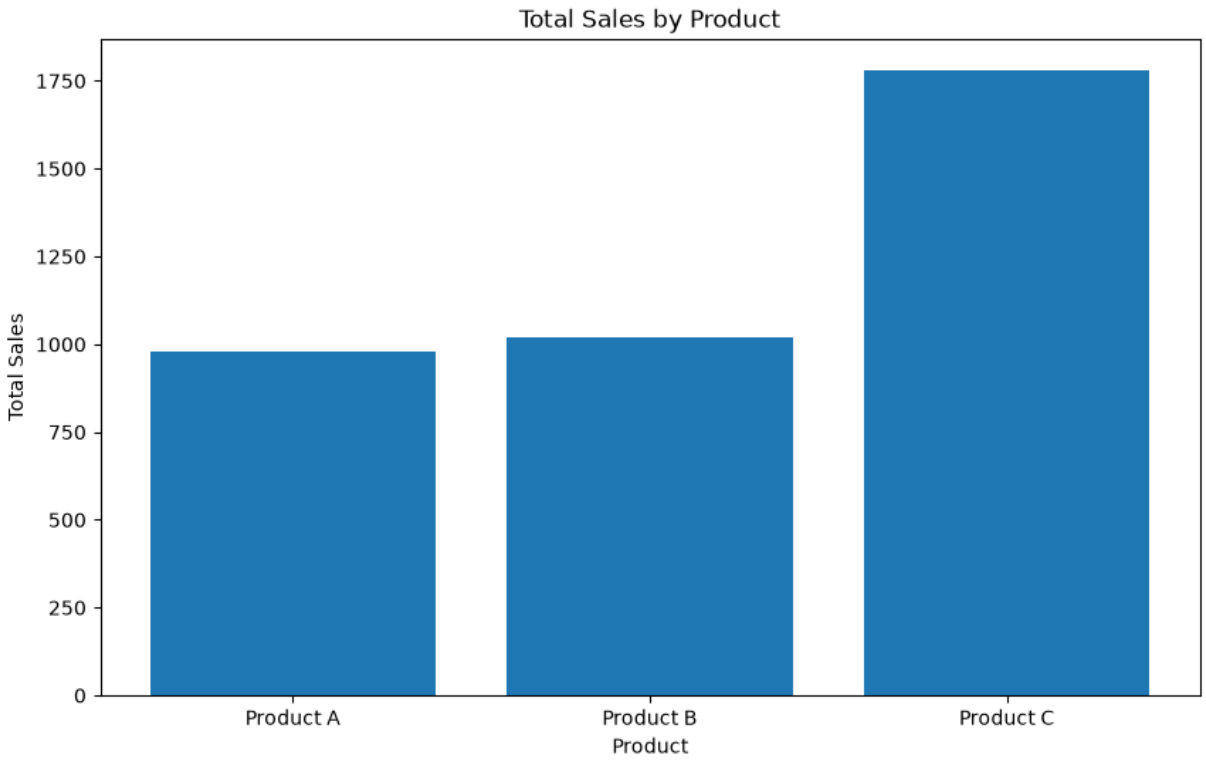
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

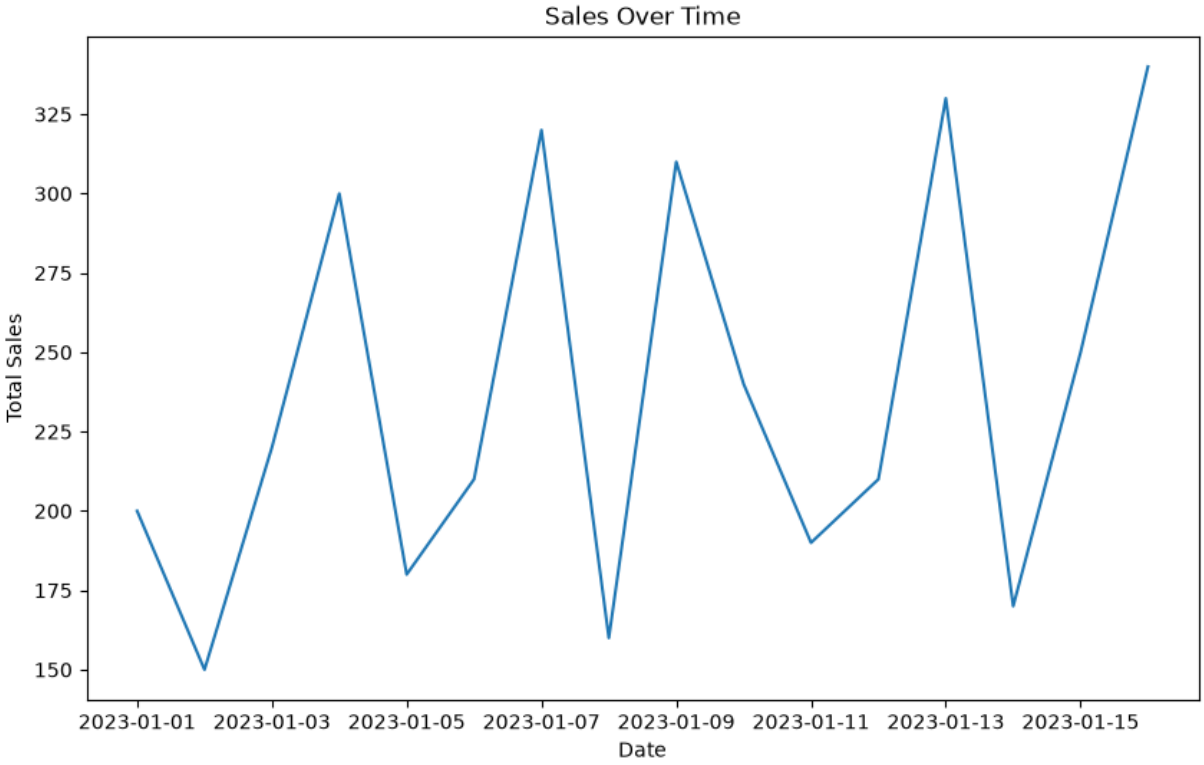
```
         Date    Product  Sales  Quantity Region
0  2023-01-01  Product A    200         4  North
1  2023-01-02  Product B    150         3  South
2  2023-01-03  Product A    220         5  North
3  2023-01-04  Product C    300         6   East
4  2023-01-05  Product B    180         4   West
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
            Sales   Quantity
count   16.000000  16.000000
mean   236.250000   5.312500
std     64.381674   1.621471
min    150.000000   3.000000
25%    187.500000   4.000000
50%    215.000000   5.000000
75%    302.500000   6.250000
max    340.000000   8.000000
     Product  Sales  Quantity
0  Product A    980        21
1  Product B   1020        23
2  Product C   1780        41
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_7516\534910707.py:16: ChainedAssignmentEr
ror: A value is being set on a copy of a DataFrame or Series through chained assignm
ent using an inplace method.
Such inplace method never works to update the original DataFrame or Series, because
the intermediate object on which we are setting values always behaves as a copy (due
to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method
({col: value}, inplace=True)' instead, to perform the operation inplace on the origi
nal object, or try to avoid an inplace operation using 'df[col] = df[col].method(val
ue)'.

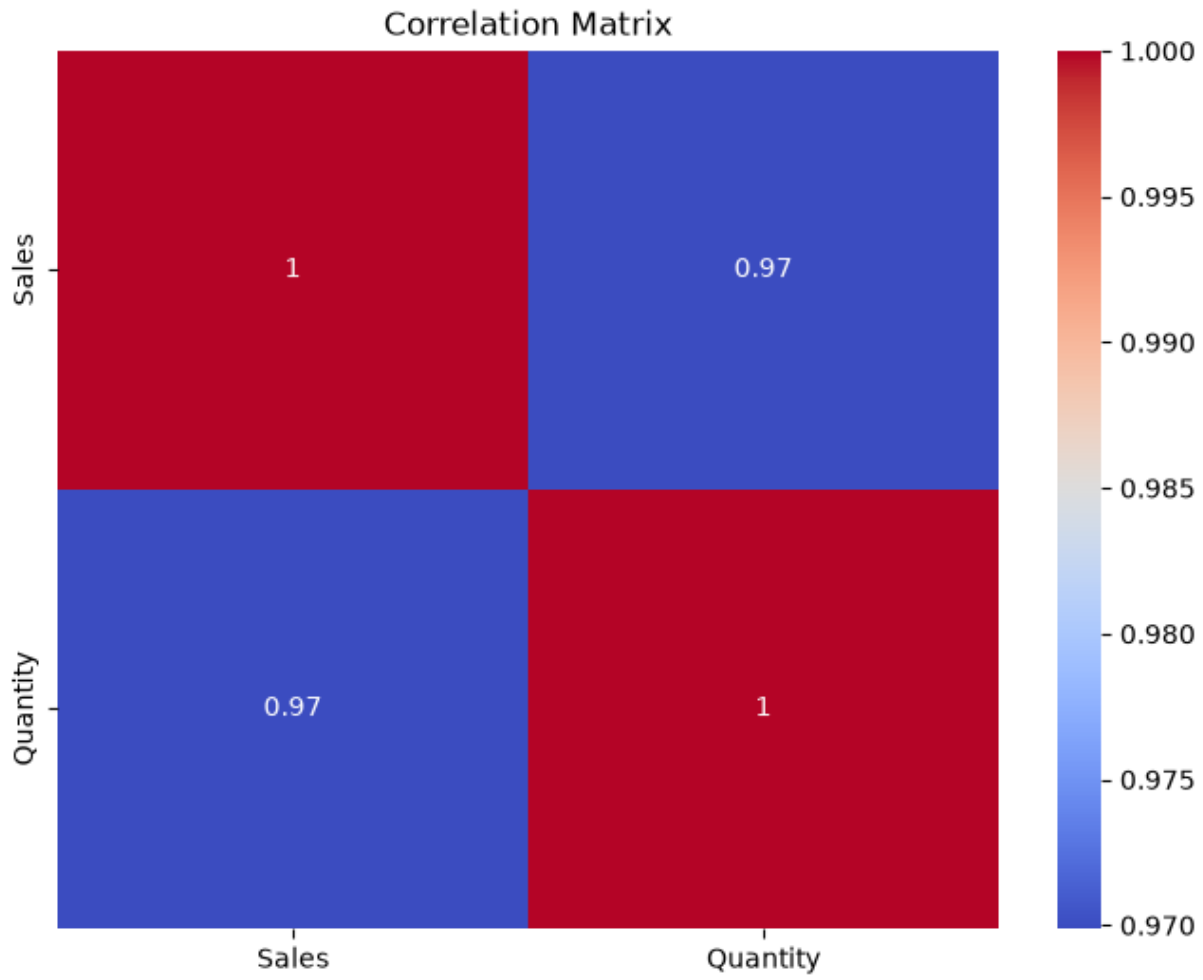
See the documentation for a more detailed explanation: https://pandas.pydata.org/pan
das-docs/stable/user_guide/copy_on_write.html
  df['Sales'].fillna(df['Sales'].mean(), inplace=True)
```





Product	Product A	Product B	Product C
Region			
East	0	0	1780
North	980	0	0
South	0	460	0
West	0	560	0

	Sales	Quantity
Sales	1.000000	0.969893
Quantity	0.969893	1.000000


Correlation Matrix
Sales
Quantity
1
0.97
0.97
1
Sales
Quantity
1.000
0.995
0.990
0.985
0.980
0.975
0.970

In [ ]: